

Edge cases (a missing required argument) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/Python), **Advanced** (what you'd actually do once you know the model sometimes guesses). Read Basic first even if you already know this — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise isn't really about writing new code — it's about watching real model behavior on purpose, so you're not surprised by it later.

Give a tool a required argument (like `city: str`), then ask a question that clearly needs the tool but never gives it a city. The model has three real options here: ask you a clarifying question back, guess a city, or refuse to call the tool at all. There's no code fix needed yet — your job is to run it several times and see which one actually happens.

Things to use:

- A tool with one required argument, no default value — `city: str`, not `city: str = "London"`.
- A prompt that needs the tool but never names a city.
- Run it in a loop (5 times is enough) and print, each time, whether a tool call happened and what arguments it carried.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Reuse a tool from an earlier exercise (`get_weather(city: str)` works fine) rather than building a new one — the point here is the *prompt*, not the tool.

Run the same ambiguous prompt (something like "what's the weather like") several times, and log, for each run: did the model call the tool at all, and if so, what value did it send for `city`? A model might send an empty string, a guessed default city, or genuinely refuse the call and respond with plain text asking you for the city.

Look specifically at `response.content` vs. `response.tool_calls`: when the model refuses to call the tool and asks you a question instead, that question lands in `response.content`, and `response.tool_calls` will be empty — check both every time, don't assume one or the other.

Three buckets are enough to classify each run: "asked a clarifying question," "called the tool with a guessed/empty value," and "called the tool with something else unexpected."

This is exactly why "checking arguments" (Core Concepts) isn't the whole story: Pydantic only checks that a `city` value *exists and is a string* — it can't tell you whether the model quietly guessed "London" as a default. That's a values problem, not a shape problem, and it's worth seeing the difference here. Write your classification function's signature before moving to Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Once you've watched the model actually guess a city, the real question isn't "what did it do" anymore — it's **"what should this tool do about it?"** Three genuinely different design answers exist, and this exercise's Basic/Intermediate levels never had to pick one:

Reject an empty guess at the validation layer, not the business-logic layer. If the model sends `city=""`, Pydantic's default check ("is this a string?") happily accepts it — an empty string *is* a string. A `Field(min_length=1)` constraint (or a small custom validator) makes an empty guess fail *at the edge*, the same way a missing field already does, instead of your tool's own code having to remember to check `if not city:` every time.

Distinguish "empty" from "confidently wrong." An empty string and a plausible-looking guess like `"London"` are different failure modes with different fixes. An empty string means the model basically refused but called anyway — validation catches that cleanly. A confident wrong guess is harder: it *looks* like a real answer, so nothing crashes and nothing looks broken, but the tool ran with input the user never actually gave. This is the more dangerous case, and it's a values problem no type system catches — you'd need a small list of "suspicious default" values (common demo cities, "N/A", "unknown") to flag it heuristically, and even that's an imperfect guess.

Remember this is a multi-caller problem. This exact tool will get imported into the Build Task's library, and from there into every agent from Doc07 onward. A silent wrong-city guess that "worked" here becomes a wrong weather report in a real conversation later, traced back to a missing argument nobody caught at the source. The fix belongs here, once, in the tool's own argument validation — not re-implemented by every future caller that happens to remember to check.

The extra piece needed for the validation-layer fix:

- A Pydantic `BaseModel` for the tool's arguments, with `city: str = Field(min_length=1)`, passed to `@tool` via `args_schema=`, instead of relying on the plain typed parameter.

Sketch this Pydantic args model yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate observe and classify what the model actually does with a missing argument — asks, guesses empty, or guesses a real-looking value. Advanced treats that observation as a design problem instead of just a finding: catching the empty-guess case at validation instead of business logic, and naming why a confident wrong guess is the harder, more dangerous case that no type system catches on its own — the same "silently wrong is worse than loudly failing" idea from Core Concepts' failure-handling section, applied here to a value instead of a crash.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define get_weather(city) as before, city has no default

register it on a model

repeat 5 times:
    ask "what's the weather like" (no city mentioned)
    if the model called the tool:
        print "called with city =", the argument it sent
    else:
        print "asked instead:", the model's text reply
```

Here is almost the whole thing:

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather])
```

What's missing: the loop of 5 runs, and printing what actually happened each time. Add those, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def classify_response(response) -> str:
    if not response.tool_calls:
        return "asked_or_answered_directly"
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"
```

Wire this into a 5-run loop yourself, printing each run's outcome (and the model's reply when it asked instead of calling), then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the missing Pydantic constraint yourself — this is the piece that turns an empty-string guess into a caught validation error instead of a silently accepted one:

```
from pydantic import BaseModel, Field, ValidationError
from langchain_core.tools import tool

class WeatherArgs(BaseModel):
    # your turn: add a `city` field, typed str, with a constraint that
    # rejects an empty string (not just a missing field)
    ...

@tool(args_schema=WeatherArgs)
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

def classify_response(response) -> str:
    if not response.tool_calls:
        return "asked_or_answered_directly"

    args_sent = response.tool_calls[0]["args"]
    try:
        WeatherArgs(**args_sent)
    except ValidationError:
        return "rejected_by_validation"

    return f"called_with_value:{args_sent.get('city')}"
```

Wire this into the same 5-run loop, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying loop (ask 5 times, classify what happened) at 3 completeness levels — Basic just prints what happened, Intermediate sorts each run into named buckets, and Advanced adds a real Pydantic constraint so an empty-string guess gets *caught*, not just *labeled* — turning "the model called with `city=''`" from an observation into an actual validation failure the tool's own contract enforces.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)